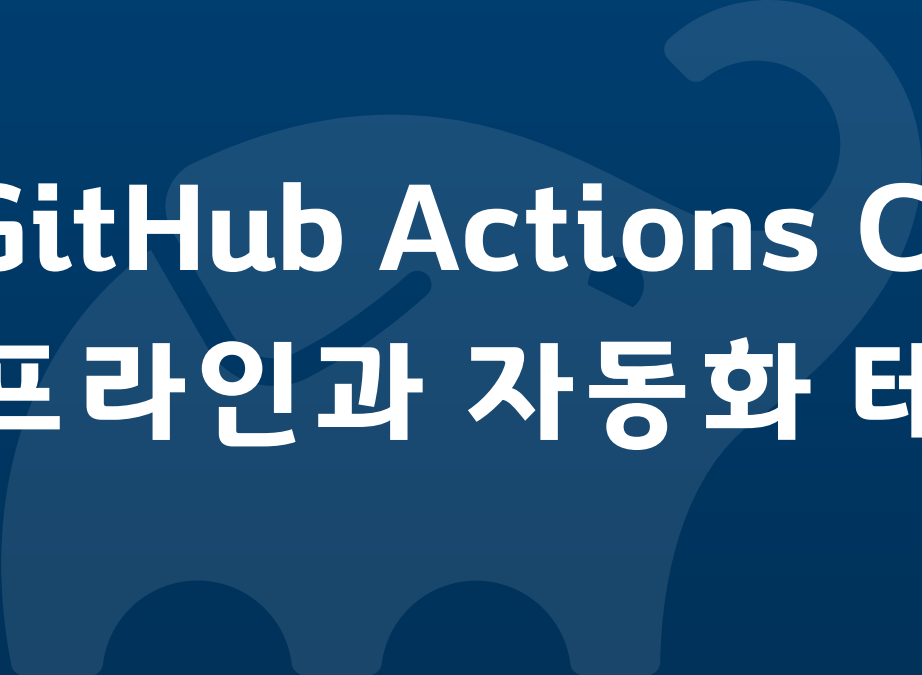




GitHub Actions CI

파이프라인과 자동화 테스트

학습 목표

- 클론한 실습 프로젝트의 단위 테스트와 슬라이스 테스트로 도메인 로직과 웹 계층을 자동 검증합니다
- Gradle Wrapper가 반환하는 종료 코드가 CI의 성공과 실패 판정 기준임을 확인합니다
- GitHub Actions 워크플로로 푸시와 Pull Request마다 테스트를 자동 실행합니다
- 브랜치 보호 규칙과 필수 상태 확인으로 결함 코드의 `main` 병합을 차단합니다
- 테스트 통과를 전제로 하는 이미지 빌드 Job을 연결해 파이프라인을 완결합니다

검증 시점을 배포 이후에서 병합 이전으로 앞당깁니다. 사람의 반복 확인이 아니라 러너가 실행하는 자동화 검증 (Assertion)이 결함의 유입 경로를 차단합니다.

핵심 개념 및 구조

품질 검증 체계의 세 단계

구분	수동 API 검증	런타임 관측과 경보	사전 자동화 검증
검증 시점	로컬 수동 호출 시	배포 후 운영 중	커밋 푸시 후 병합 전
수행 주체	사람의 브라우저 클릭	Prometheus, Loki	격리된 Ubuntu 러너
검증 대상	단일 엔드포인트 응답	자원 사용률과 에러율	도메인 로직과 기대값 검증
결함 발견 비용	큼 (릴리스 직전 발견)	매우 큼 (사용자 노출 후)	가장 낮음 (수 분 내 피드백)

관측 스택은 장애를 인지하고 원인을 추적하는 도구이지 결함의 유입을 막는 도구가 아닙니다. 결함 차단은 병합 이전 단계에서만 가능합니다.

CI의 3대 기둥

📁 단일 소스 저장소

팀 전체가 신뢰하는 기준 브랜치 하나를 공유합니다

⚡ 빌드와 테스트 자동화

커밋마다 격리된 러너가 동일한 절차를 실행합니다

🛑 조기 실패와 즉각 피드백

실패 즉시 중단하고 결함 발견 시점을 앞당깁니다

기둥	실습에서의 구현체
단일 소스 저장소	<code>main</code> 브랜치와 브랜치 보호 규칙
빌드와 테스트 자동화	<code>.github/workflows/ci.yml</code> 의 <code>test</code> Job
조기 실패와 즉각 피드백	필수 상태 확인과 머지 버튼 비활성화

테스트 유형: 유닛 테스트 · 통합 테스트 · 스모크 테스트

🔗 유닛 테스트

Unit Test · 수 밀리초

- 외부 의존성(DB, 네트워크)을 Mock으로 전면 격리
- 순수 도메인 비즈니스 로직 단위 초고속 검증
- 실행 대상: `UserServiceTest`

🔗 통합 테스트

Integration Test · 수 초

- 스프링 컨텍스트 로딩 및 컴포넌트 간 상호작용 검증
- Controller-Service-DB 계층 연동 검증
- 실행 대상: `UserControllerTest`

🔥 스모크 테스트

Smoke Test · 수 초 ~ 수 분

- 배포·기동 직후 핵심 기능의 정상 동작 신속 점검
- 헬스체크 엔드포인트 및 필수 API 최소 검증
- 실행 대상: 컨테이너 기동 후 상태 점검

단위 테스트로 도메인 결함을 신속히 차단하고, 통합·스모크 테스트로 계층 간 연동과 배포 안정성을 단계적으로 검증합니다.

테스트 주도 개발(TDD)과 행위 주도 개발(BDD)

🔗 테스트 주도 개발 (TDD)

개발자 관점 · 내부 설계와 코드 품질 중심

- **Red**: 실패하는 단위 테스트를 먼저 작성
- **Green**: 테스트를 통과하는 최소한의 프로덕션 코드 구현
- **Refactor**: 구현 코드와 테스트의 가독성·중복을 개선

결함 사전 예방과 안전한 리팩터링 안전망을 확보합니다.

👥 행위 주도 개발 (BDD)

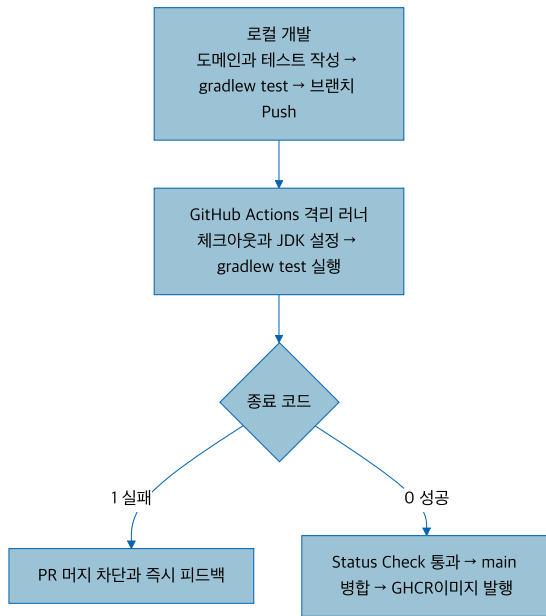
협업 관점 · 비즈니스 요구사항 명세 중심

- **Given**: 테스트 사전 상태와 환경 조건 설정
- **When**: 검증 대상 동작이나 비즈니스 이벤트 실행
- **Then**: 기대하는 결과값 및 상태 변화 검증

사용자 관점의 시나리오를 통해 이해관계자 소통을 촉진합니다.

TDD 사이클로 코드 품질과 설계 안전성을 확보하고, BDD Given-When-Then 패턴으로 요구사항 명세의 가독성을 높입니다.

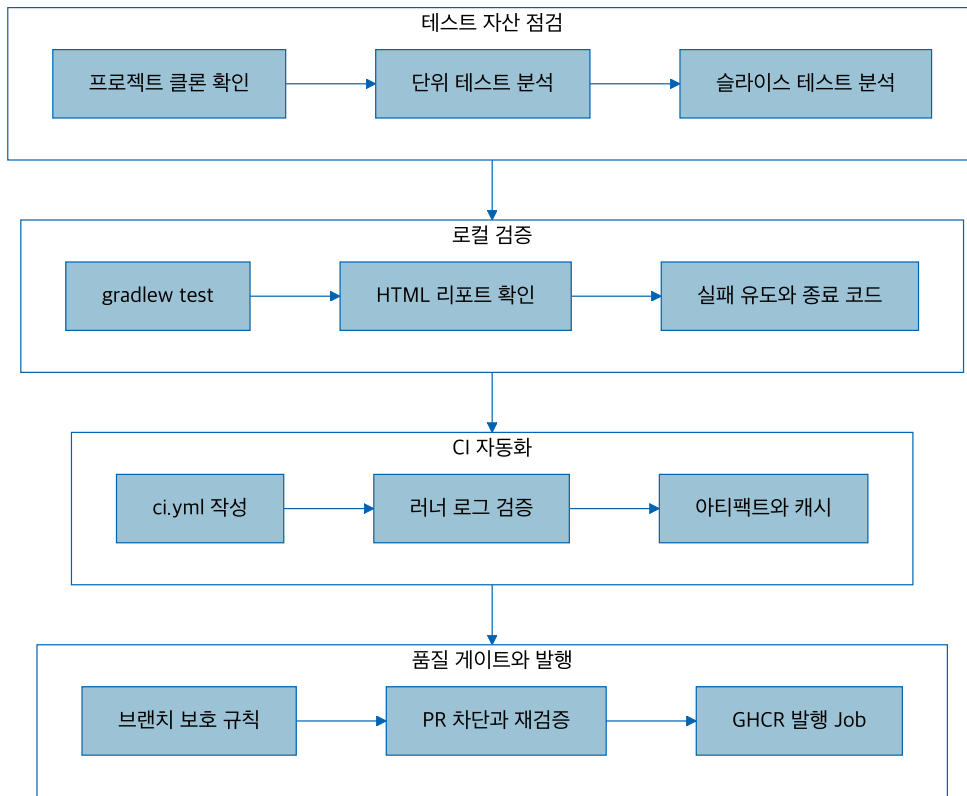
커밋에서 이미지 발행까지의 파이프라인



러너는 표준 출력 문자열이 아닌 프로세스 종료 코드(Exit Code 0 vs 1)로 판정하며, 비정상 종료 시 즉시 중단하는 Fail-Fast가 CI 파이프라인의 핵심 기준입니다.

실습 단계

실습 전체 흐름: 테스트 작성부터 이미지 자동 발행까지



실습 프로젝트 클론과 테스트 자산 점검

```
cd ~/workspace/simple-back

# 테스트 의존성 확인
grep -n "starter-test\|webmvc-test\|h2database" build.gradle

# 저장소에 포함된 테스트 클래스 확인
find src/test -type f -name "*.java" | sort
```

5 JUnit 5

테스트 메서드 선언과 실행 엔진을 제공합니다



AssertJ

가독성 높은 검증(Assertion) API를 제공합니다



Mockito

외부 의존성을 대체할 모의 객체를 생성합니다

세 라이브러리는 `spring-boot-starter-test` 의존성에 함께 포함되며, 슬라이스 테스트는 `spring-boot-starter-webmvc-test`가 추가로 필요합니다.

저장소에 포함된 테스트 자산

테스트 클래스	격리 방식	검증 내용
<code>app/UserServiceTest</code>	Mockito <code>@Mock</code>	데이터가 비었을 때 기본 사용자 저장 여부
<code>ui/UserControllerTest</code>	<code>@WebMvcTest</code> + <code>@MockitoBean</code>	<code>GET /</code> , <code>GET /users</code> 응답 상태와 JSON
<code>SimpleBackApplicationTests</code>	<code>@SpringBootTest</code> + H2	전체 스프링 컨텍스트 로딩 성공 여부

본 차시는 애플리케이션 코드를 새로 작성하지 않고, 클론한 프로젝트의 기존 테스트를 CI 파이프라인에 연결합니다. 계층 구조와 프로파일 설계는 실습 프로젝트 명세 문서에 정리되어 있습니다.

비즈니스 로직 단위 테스트

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @Test
    void createsDefaultUserWhenRepositoryIsEmpty() {
        given(userRepository.count()).willReturn(0L);
        given(userRepository.findAll()).willReturn(List.of());
        UserService userService = new UserService(userRepository);

        userService.getUsers();

        ArgumentCaptor<User> userCaptor =
            ArgumentCaptor.forClass(User.class);
        verify(userRepository).save(userCaptor.capture());
        assertThat(userCaptor.getValue().email())
            .isEqualTo("admin@example.com");
    }
}
```

구성 요소	기능
@ExtendWith	Mockito 확장을 JUnit 5에 활성화
@Mock	저장소 포트를 모의 객체로 대체
given().willReturn()	호출 시 반환값 스텀빙 정의
ArgumentCaptor	전달된 인자를 포획해 내용 검증
verify()	메서드 호출 여부 상호작용 검증

Web 계층 슬라이스 테스트

```
@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired
    private MockMvc mockMvc;
    @MockitoBean
    private UserQueryUseCase userQueryUseCase;

    @Test
    @DisplayName("루트 경로 요청 시 HTTP 200과 상태 메시지를 반환한다")
    void testRootEndpoint() throws Exception {
        mockMvc.perform(get("/"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.status").value("UP"));
    }

    @Test
    @DisplayName("사용자 목록 조회 시 HTTP 200과 목록을 반환한다")
    void testGetUsersEndpoint() throws Exception {
        given(userQueryUseCase.getUsers()).willReturn(Collections.emptyList());

        mockMvc.perform(get("/users"))
            .andExpect(status().isOk())
            .andExpect(content().contentType("application/json"));
    }
}
```

슬라이스 테스트 구성 요소

구성 요소	소속 기술	기능
<code>@WebMvcTest</code>	Spring Boot Test	웹 계층 빈만 선별 로드하여 컨트롤러 검증
<code>@MockitoBean</code>	Spring Test	유스케이스 빈을 모의 객체로 대체해 주입
<code>MockMvc</code>	Spring Test	서블릿 컨테이너 없이 가상 HTTP 요청 수행
<code>given().willReturn()</code>	BDD Mockito	특정 인자 호출 시 반환값 스테빙 정의
<code>jsonPath()</code>	Jayway JsonPath	응답 JSON 본문의 키와 값 검증

`@WebMvcTest`는 네트워크 포트를 바인딩하지 않으므로 포트 충돌 없이 실행됩니다. Spring Boot 4에서는 import 경로가 `org.springframework.boot.webmvc.test.autoconfigure`로 바뀌고 `spring-boot-starter-webmvc-test` 의존성이 필요합니다.

로컬 테스트 실행과 리포트 확인

```
./gradlew test

ls -la build/reports/tests/test/index.html

# 브라우저로 리포트 열기
open build/reports/tests/test/index.html
```

📄 리포트 구성

전체 테스트 수, 성공률, 소요 시간, 클래스별 상세 케이스 목록

산출물 경로

용도

`build/reports/tests/test/`

사람이 읽는 HTML
대시보드

`build/test-results/test/`

외부 도구가 파싱하는
JUnit XML

`build/classes/java/test/`

컴파일된 테스트 클래스

터미널 마지막 줄에 `BUILD SUCCESSFUL`이 출력되고
`index.html`이 생성되었는지 확인합니다.

의도적 실패 유도과 종료 코드 확인

```
# 기대값을 틀리게 수정해 실패 유도
sed -i.bak 's/"admin@example.com"/"wrong@example.com"/g' \
  src/test/java/org/example/simpleback/app/UserServiceTest.java
./gradlew test
echo $?

# 원래 코드로 복원 후 재실행
mv src/test/java/org/example/simpleback/app/UserServiceTest.java.bak
  src/test/java/org/example/simpleback/app/UserServiceTest.java
./gradlew test
echo $?
```

프로세스 상태	종료 코드	러너 동작
BUILD FAILED	1	파이프라인 즉시 중단, 실패 처리
BUILD SUCCESSFUL	0	후속 Job 진행

⚠ Fail-Fast의 근거

셸의 모든 명령은 정상 종료 시 0, 오류 시 1 이상을 반환합니다. CI도구는 이 값만 읽고 판정합니다.

CI 워크플로 정의: 트리거와 실행 환경

```
name: Continuous Integration

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  test:
    name: Run Unit & Slice Tests
    runs-on: ubuntu-latest
```

🔗 트리거 조건

`main` 을 타깃으로 하는 푸시와 Pull Request 생성 및 갱신 시 실행됩니다

▶ 실행 환경

GitHub이 매 실행마다 새 Ubuntu가상 머신을 프로비저닝합니다

Job이름 `Run Unit & Slice Tests` 는 뒤에서 필수 상태 확인 대상으로 지정할 식별자입니다.

CI 워크플로 정의: 테스트 실행과 리포트 보관

```
steps:
  - name: Checkout repository
    uses: actions/checkout@v6

  - name: Set up JDK 17
    uses: actions/setup-java@v6
    with:
      java-version: '17'
      distribution: 'temurin'
      cache: 'gradle'

  - name: Grant execute permission for gradlew
    run: chmod +x gradlew

  - name: Run tests with Gradle Wrapper
    run: ./gradlew test

  - name: Upload test results
    if: always()
    uses: actions/upload-artifact@v7
    with:
      name: gradle-test-reports
      path: build/reports/tests/test/
```

스텝	역할
checkout	러너에 저장소 소스 배치
setup-java	Temurin JDK 17 설치와 의존성 캐시
chmod +x gradlew	래퍼 실행 권한 누락 차단
upload-artifact	HTML 리포트를 zip으로 보관

◆ 조건부 실행

`if: always()` 는 테스트가 실패해도 리포트 업로드를 수행하게 합니다

커밋 푸시와 러너 실행 로그 검증

```
git status
git add src/ .github/
git commit -m "feat: add user domain, tests and ci workflow"
git push origin main
```

저장소 [Actions](#) 탭에서 [Continuous Integration](#) 워크플로를 선택하고 방금 푸시한 커밋으로 트리거된 실행을 엽니다. Job 내부의 각 스텝이 순차 실행되는 과정을 확인합니다.

확인 영역

정상 판정 기준

워크플로 상태
아이콘

진행 중 표시에서 녹색 체크로 전환

[Run tests](#) 로그

[BUILD SUCCESSFUL](#) 출력

전체 Job 수행

의존성 다운로드 포함 수 분 이내
완료

[Set up JDK 17](#) 스텝 로그를 펼쳐 Temurin JDK 17이 러너에 설치되는지 확인합니다.

테스트 아티팩트와 Gradle 캐시 확인

1. Actions 실행 상세 페이지의 **Summary** 화면 하단으로 이동합니다
2. **Artifacts** 항목의 **gradle-test-reports** zip 파일을 내려받습니다
3. 압축을 풀고 **index.html** 을 열어 로컬 리포트와 결과가 같은지 확인합니다
4. **Post Set up JDK 17** 스텝을 펼쳐 캐시 저장 로그를 확인합니다

항목	기능
gradle-test-reports	러너 종료 시 사라질 리포트를 보관
Post Set up JDK 17	~/.gradle/caches 를 해시 기반으로 저장

캐시가 적중하는 두 번째 실행부터는 의존성 다운로드가 생략되어 실행 시간이 줄어듭니다. 아티팩트 보관 기간은 기본 90일입니다.

브랜치 보호 규칙과 필수 상태 확인 구성

저장소 Settings → Branches → Add branch protection rule에서 설정합니다.

- Branch name pattern: `main`
- Require a pull request before merging 체크
- Require status checks to pass before merging 체크
- 검색창에서 `Run Unit & Slice Tests`를 선택
- Do not allow bypassing the above settings 체크

설정 항목	통제 효과
<code>Require a pull request</code>	<code>main</code> 직접 푸시 차단
<code>Require status checks</code>	CI 실패 시 머지 버튼 비활성화
<code>Do not allow bypassing</code>	관리자 권한의 규칙 우회 차단

혼자 진행하는 실습 환경이므로 `Require approvals`는 해제합니다. 저장 후 목록에서 `main` 옆에 `Protected` 배지가 표시되는지 확인합니다.

보호된 main 브랜치 직접 푸시 차단 검증

```
# 빈 테스트 커밋 생성 후 main 직접 푸시 시도 (차단되어야 정상)
git commit --allow-empty -m "test: direct push to protected main branch"
git push origin main

# 시도했던 로컬 빈 커밋 되돌리기
git reset --hard HEAD~1
```

원격 서버가 다음 두 줄을 반환하며 푸시를 거부합니다.

```
remote: error: GH006: Protected branch update failed for refs/heads/main.
```

```
remote: error: Changes must be made through a pull request.
```

관찰 항목	결과	의미
저장소 소유자 계정의 직접 푸시	원격에서 거부	<code>Do not allow bypassing</code> 이 관리자에게도 적용됨
<code>git reset --hard HEAD~1</code>	로컬 임시 커밋 제거	로컬 저장소를 <code>origin/main</code> 과 동일 상태로 복원

피쳐 브랜치 분기와 실패 테스트 작성

```
// src/test/java/org/example/simpleback/app/
// UserServiceDefaultUserTest.java
@ExtendWith(MockitoExtension.class)
class UserServiceDefaultUserTest {
    @Mock
    private UserRepository userRepository;

    @Test
    @DisplayName("신규 요구사항: 기본 사용자 이름 검증")
    void checkDefaultUserName_Failure() {
        given(userRepository.count()).willReturn(0L);
        given(userRepository.findAll()).willReturn(List.of());
        new UserService(userRepository).getUsers();

        ArgumentCaptor<User> userCaptor =
            ArgumentCaptor.forClass(User.class);
        verify(userRepository).save(userCaptor.capture());
        // 실제 저장값은 infra-admin이므로 검증이 실패한다
        assertThat(userCaptor.getValue().name())
            .isEqualTo("vip-admin");
    }
}
```

🔗 작업 공간 격리

`git checkout -b feature/user-update`로 `main`과 분리된 브랜치에서
변경을 누적합니다.

⊗ 의도적 검증 실패

기대값을 `vip-admin`으로 지정해 Assertion Error를 유발하고 품질 게이트의
차단 동작을 확인합니다. 새 테스트는 컴포넌트 스캔 기준 패키지 하위에
둡니다.

실패 테스트 푸시와 Pull Request 생성

```
git checkout -b feature/user-update
git add src/test/java/org/example/simpleback/app/UserServiceDefaultUserTest.java
git commit -m "test: add default user name verification (broken assertion)"
git push -u origin feature/user-update
```

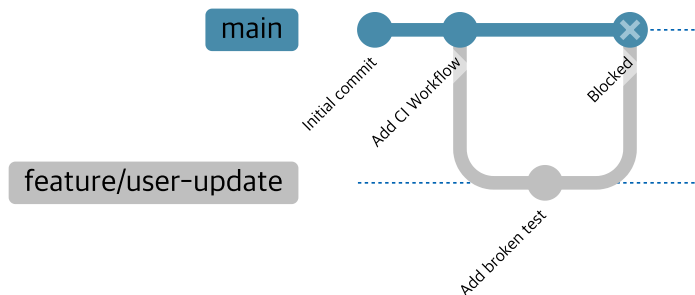
🔗 PR 생성 절차

저장소 상단의 `Compare & pull request`를 클릭하고 Base를 `main`, Compare를 `feature/user-update`로 확인한 뒤 `Create pull request`를 누릅니다.

📄 자동 트리거 확인

PR이 열리는 즉시 `pull_request` 이벤트가 발생해 `Continuous Integration` 워크플로가 시작됩니다.

Status Check 실패와 머지 차단



항목	화면 표시	통제 원리
Checks	빨간색 X	테스트 실패로 종료 코드 1 반환
Merge 버튼	비활성화	필수 상태 확인 미충족으로 병합 차단
피드백	Details 로그	실패 원인을 PR 화면에서 즉시 확인

Required statuses must pass before merging
경고와 함께 `UserServiceDefaultUserTest > checkDefaultUserName_Failure FAILED` 로그를
확인합니다.

검증 로직 수정과 로컬 사전 검증

```
@ExtendWith(MockitoExtension.class)
class UserServiceDefaultUserTest {

    @Mock
    private UserRepository userRepository;

    @Test
    @DisplayName("신규 요구사항: 기본 사용자 이름 검증")
    void checkDefaultUserName_Success() {
        given(userRepository.count()).willReturn(0L);
        given(userRepository.findAll()).willReturn(List.of());
        new UserService(userRepository).getUsers();

        ArgumentCaptor<User> userCaptor =
            ArgumentCaptor.forClass(User.class);
        verify(userRepository).save(userCaptor.capture());
        // 실제 저장값에 일치하도록 검증문(Assertion) 수정
        assertThat(userCaptor.getValue().name())
            .isEqualTo("infra-admin");
    }
}
```

```
# 원격 푸시 전 로컬에서 먼저 검증
./gradlew test
```

```
git add src/test/java/org/example/simpleback/app/UserServiceDefaultUserTest.java
git commit -m "fix: correct expected default user name assertion"
git push origin feature/user-update
```

로컬 선검증

러너 실행 시간을 소모하기 전에 동일한 Gradle 태스크로 결과를 먼저 확인합니다.

PR 재검증과 main 안전 병합

단계	관찰 현상	시스템 동작 원리
수정 푸시	러너 자동 재실행	PR 브랜치에 커밋이 추가될 때마다 상태 점검을 다시 수행
테스트 성공	Checks 녹색 전환	모든 검증 조건을 만족해 종료 코드 0 반환
머지 승인	Merge 버튼 활성화	보호 규칙의 필수 상태 확인 조건을 모두 충족
병합 완료	PR 종료 및 머지 커밋 생성	검증된 코드만 <code>main</code> 에 반영

⚙️ All checks have passed

문구가 표시되면 `Merge pull request`와 `Confirm merge`로 병합을 확정합니다.

🔍 병합 결과 확인

커밋 히스토리에서 `Merge pull request #1 from ...` 머지 커밋 생성을 확인합니다.

워크플로 확장: 테스트 Job 정의

```
name: Continuous Integration and Package
on:
  push: { branches: [ "main" ] }
  pull_request: { branches: [ "main" ] }
env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${github.repository}
jobs:
  test:
    name: Run Unit and Slice Tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-java@v6
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: 'gradle'
      - run: chmod +x gradlew
      - run: ./gradlew test
      - if: always()
        uses: actions/upload-artifact@v7
        with:
          name: gradle-test-reports
          path: build/reports/tests/test/
```

지시어	역할
<code>env</code>	레지스트리와 이미지명 전역 선언
<code>cache: gradle</code>	의존성 캐시로 빌드 시간 단축
<code>chmod +x</code> <code>gradlew</code>	래퍼 실행 권한 부여
<code>if: always()</code>	실패 시에도 리포트 업로드

Job이름 `test`가 필수 상태 확인 대상과 일치해야 머지 차단이 동작합니다.

GHCR 푸시 Job 정의

```
docker-build-push:
  name: Build and Push Image to GHCR
  needs: test
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write
  steps:
    - uses: actions/checkout@v6
    - uses: docker/setup-buildx-action@v4
    - uses: docker/login-action@v4
      with:
        registry: ${ env.REGISTRY }
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }
    - run: echo "IMAGE_NAME=${IMAGE_NAME,,}" >> $GITHUB_ENV
    - uses: docker/build-push-action@v7
      with:
        context: .
        push: true
        platforms: linux/amd64,linux/arm64
        tags: |
          ${ env.REGISTRY }/${ env.IMAGE_NAME }:latest
          ${ env.REGISTRY }/${ env.IMAGE_NAME }:${ github.sha }
```

지시어	역할
<code>needs: test</code>	<code>test</code> 가 성공해야만 빌드 Job 실행
<code>if</code> 조건문	PR에서는 건너뛰고 <code>main</code> 푸시에서만 푸시
<code>packages:</code> <code>write</code>	내장 토큰에 GHCR 업로드 권한 부여
<code>IMAGE_NAME,,</code>	이미지명 소문자 규칙 준수

`latest`와 커밋 해시 태그를 함께 발행해 배포 이미지의 소스 버전을 추적합니다.

워크플로 변경 PR과 Job 실행 조건 검증

```
git checkout main && git pull origin main
git checkout -b chore/ci-docker-pipeline
git add .github/workflows/ci.yml
git commit -m "ci: add docker build and ghcr push job with needs dependency"
git push -u origin chore/ci-docker-pipeline
```

브랜치와 이벤트	실행되는 Job	사유
<code>chore/ci-docker-pipeline</code> PR	<code>test</code> 만 실행되고 빌드 Job은 <code>Skipped</code>	<code>if</code> 조건이 미검증 PR 코드의 레지스트리 유입을 차단
<code>main</code> 푸시 (머지 완료)	<code>test</code> 다음 <code>docker-build-push</code> 연속 실행	테스트 통과 후에만 배포 이미지 발행

➤ Actions 파이프라인의 DAG (Directed Acyclic Graph, 방향성 비순환 그래프) 뷰에서 두 Job 사이의 의존성 화살표를 확인합니다.

GHCR 멀티 플랫폼 이미지 태그 확인

확인 항목	기대값	설명
패키지 명칭	<code>ghcr.io/<사용자명>/<리포지토리명></code>	GitHub Container Registry 공식 참조 경로
태그 목록	<code>latest</code> 와 커밋 해시	배포용 기본 태그와 불변 추적 태그를 함께 발행
지원 플랫폼	<code>linux/amd64</code> , <code>linux/arm64</code>	x86 클라우드와 ARM 인스턴스를 모두 포함한 매니페스트

🔍 콘솔 확인 경로

저장소 메인 화면의 Packages 섹션 또는 프로필의 Packages 탭에서 패키지 상세를 엽니다.

☁ 로컬 다운로드 검증

`docker pull ghcr.io/<소유자>/<이미지명>:latest` 로 러너가 빌드한 이미지를 내려받습니다.

임시 브랜치 정리와 동기화 점검

```
git checkout main
git pull origin main

git branch -d feature/user-update
git branch -d chore/ci-docker-pipeline
git push origin --delete feature/user-update || true
git push origin --delete chore/ci-docker-pipeline || true

git branch -v
git status
```

명령	기능	확인 기준
<code>git branch -d</code>	머지 완료 브랜치만 안전 삭제	미머지 브랜치는 삭제되지 않고 경고 반환
<code>git push origin --delete</code>	원격 브랜치 포인터 제거	<code>git branch -a</code> 결과에 임시 브랜치가 없음

 최종 상태는 로컬과 원격 모두 `main` 만 남고 `working tree clean` 이어야 합니다.

Appendix

테스트 피라미드와 계층별 비용

테스트 계층	대상 범위	외부 의존성 격리	실행 속도	작성 및 유지 비용
E2E · 스모크	사용자 관점 전 구간	실제 DB · 외부 API 연동	수 분 이상	매우 높음
통합 · 슬라이스	계층 간 연동 및 Web API	모의 객체 또는 인메모리 DB	수 초	보통
단위 (Unit)	단일 클래스 비즈니스 로직	Mockito로 외부 의존성 100% 차단	수 밀리초	매우 낮음

마이크 콘(Mike Cohn)의 테스트 피라미드 모델은 저비용·초고속 단위 테스트를 두텁게 구축하고 상위 계층으로 갈수록 최소한의 핵심 경로만 검증하여 전체 피드백 속도와 유지 비용을 최적화하는 원칙입니다.

CI도구 아키텍처와 운영 비용 비교

비교 항목	GitHub Actions	Jenkins
호스팅 형태	클라우드 완전 관리형 SaaS	사내 서버·EC2 자체 호스팅
초기 구축 · 운영	YAML 등록 즉시 구동 (운영 부담 없음)	서버 프로비저닝·보안 패치·플러그인 관리
파이프라인 정의	<code>.github/workflows/*.yaml</code> (선언적 YAML)	<code>Jenkinsfile</code> (Groovy 파이프라인 DSL)
러너 확장성	클라우드 호스팅 러너 자동 할당	쿠버네티스 에이전트 또는 고정 슬레이브 구성
비용 모델	월 무료 분량 제공, 분당 종량 과금	인스턴스 상시 가동비 및 유지보수 공수

GitHub Marketplace 액션 재사용과 저장소 이벤트 통합이 강점이며, Jenkins는 1,800여 개 플러그인으로 사내 맞춤 인프라 요구를 유연하게 흡수합니다.

폐쇄망 환경과 SaaS CI의 선택 기준

Jenkins의 위상

2004년 Hudson 프로젝트에서 출발해 2011년 오픈소스로 독립했습니다. 금융, 국방, 공공처럼 인터넷과 단절된 망분리 환경에서는 소스코드가 외부 SaaS로 나갈 수 없어 사내 IDC나 전용 VPC 내부 구축이 사실상 표준입니다.

GitHub Actions의 부상

2019년 정식 출시된 저장소 네이티브 CI/CD 도구로 서버 구축과 OS 패치, 백업 없이 즉시 사용됩니다. PR, Push, Release 등 저장소 이벤트와 직접 결합되어 협업 흐름이 단절 없이 이어집니다.

선택 기준은 도구의 성능이 아니라 데이터 주권과 망분리 규제입니다. 소스코드와 자격 증명의 외부 반출이 금지된 환경에서는 에어갭 구동이 가능한 Jenkins가 유일한 선택지가 됩니다.

판단 기준	선택
인터넷 연결이 가능하고 저장소가 GitHub에 있음	GitHub Actions, 러너와 캐시를 위임하고 워크플로만 관리
망분리 규제나 전용 하드웨어 연동이 필요함	Jenkins, 사내 IDC에 설치하고 플러그인으로 확장

코드 커버리지 계측과 정적 분석 품질 게이트

```
plugins {  
    id 'jacoco'  
}  
  
jacocoTestCoverageVerification {  
    violationRules {  
        rule {  
            limit {  
                // 라인 커버리지 80% 미달 시 빌드 실패  
                minimum = 0.80  
            }  
        }  
    }  
}
```



바이트코드 계측으로 라인 커버리지와 조건 분기 커버리지를 측정하고 HTML 리포트로 출력합니다. 최소 기준 미달 시 빌드를 실패시킵니다.



코드를 실행하지 않고 정적 분석해 Bugs, Vulnerabilities, Code Smells, Duplications를 측정하고, 품질 게이트 미달 시 파이프라인에서 병합을 반려합니다.

품질 게이트 기준

미달 시 파이프라인 동작

라인 커버리지 최소 **0.80**

`jacocoTestCoverageVerification` 태스크가 빌드 실패 처리

신규 코드의 Blocker와 Critical 결함 0건

정적 분석 게이트가 상태를 실패로 반환해 병합 차단

기능 테스트와 비기능 부하 테스트

구분	기능 테스트	비기능 부하 테스트
검증 목적	단일 요청이 올바른 결과를 반환하는가	대량 동시 접속에서 목표 지연 시간을 유지하는가
주요 결함 영역	조건 분기 누락, 계산 오차, API 명세 불일치	커넥션 풀 고갈, 스레드 데드락, GC 정지, CPU 스로틀링
수행 시점	커밋과 PR 생성 시마다 수 초에서 수 분	릴리스 전이나 야간 정기 실행으로 수 분에서 수 시간
연계 도구	JUnit 5, AssertJ, Mockito, Actions 러너	Grafana k6, Locust, JMeter와 Prometheus 및 Grafana



단위 테스트와 슬라이스 테스트가 모두 통과해도 동시성 병목과 자원 고갈은 탐지되지 않으므로 별도의 부하 검증이 필요합니다.

부하 테스트 도구 비교

비교 항목	Grafana k6	Locust	Apache JMeter
시나리오 언어	JavaScript	Python	GUI 설정과 Groovy
동시성 모델	Go 고루틴 비동기	<code>gevent</code> 코루틴	OS 스레드
단일 노드 효율	매우 높음	보통에서 높음	낮음
분산 구성	쿠버네티스 오퍼레이터	Master-Worker 내장	Master-Slave RMI
CI 연계	최상, 종료 코드 반환	우수, 헤드리스 CLI	보통, 리포트 파싱

Grafana k6

개발자 주도 CI 연계와 빠른 피드백에 적합합니다.

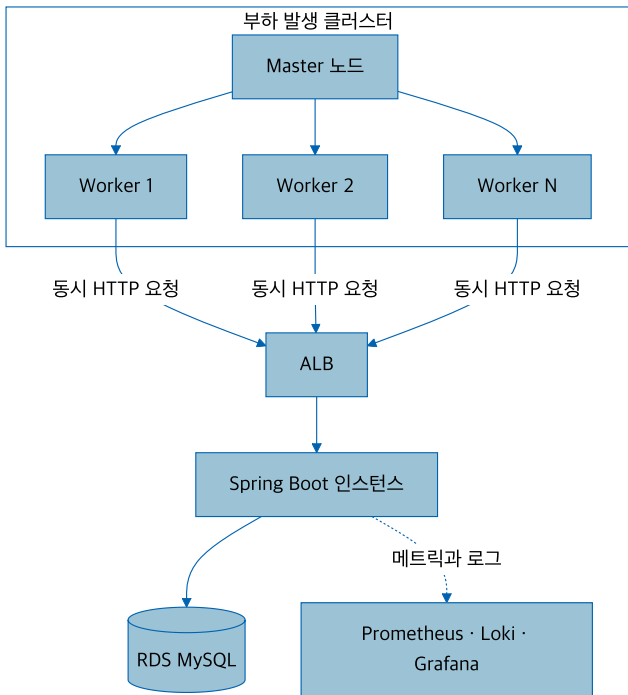
Locust

복잡한 사용자 행동 흐름을 Python으로 모델링합니다.

Apache JMeter

JDBC, JMS 등 레거시 프로토콜과 GUI 설계에 강합니다.

분산 부하 발생과 모니터링 연계



단일 부하기의 한계

임시 포트가 `TIME_WAIT`로 고갈되거나 부하기 자체의 CPU와 NIC 대역폭이 포화되어 대상 서버 성능이 왜곡 측정됩니다.

병목 상관관계 분석

지연이 급증하는 시점에 `hikaricp_connections_active` 포화, GC 정지 시간, CPU 사용률을 함께 확인해 병목 지점을 특정합니다.

성능 회귀 방지를 위한 CI 연계

임계치 항목	설정 예시	차단 대상
응답 시간	<code>http_req_duration: p(95)<400</code>	불필요한 N+1 쿼리나 무거워진 연산의 유입
오류율	<code>http_req_failed: rate<0.01</code>	부하 상태에서만 드러나는 간헐적 실패
실행 시점	PR 머지 전 스모크 부하	가상 사용자 30명 규모의 단시간 주요 API 호출



성능 예산을 코드로 선언하면 미달 시 CLI가 비정상 종료 코드를 반환하므로, 기능 테스트와 동일한 방식으로 머지를 차단할 수 있습니다.

핵심 기술 면접 질문

로컬 실행과 CI 러너

`./gradlew test`를 로컬에서 실행하는 것과 CI 러너에서 실행하는 것의 본질적 차이는 무엇인가?

필수 상태 확인 부재

필수 상태 확인을 강제하지 않으면 팀 프로젝트에서 어떤 장애 시나리오가 발생하는가?

Mock과 Spy

Mockito의 Mock 객체와 Spy 객체는 무엇이 다르며 각각 어떤 상황에 적합한가?

커버리지의 한계

커버리지가 100%인데도 운영 환경에서 장애가 발생할 수 있는 이유는 무엇인가?

핵심 기술 면접 질문

Jenkins를 고수하는 이유

대규모 엔터프라이즈가 GitHub Actions 대신 Jenkins를 유지하거나 혼용하는 보안 및 인프라적 이유는 무엇인가?

CI Green이후의 장애

테스트를 모두 통과한 애플리케이션이 대규모 트래픽에서 다운되는 원인과 분산 부하 테스트 구축 전략은 무엇인가?

모범 답변

로컬 실행과 CI 러너

로컬 PC는 OS 버전, JDK 패치, 환경변수, 경로 구분자가 개발자마다 다르므로 로컬에서만 통과하는 환경 불일치가 발생한다. CI 러너는 커밋마다 격리된 표준 Ubuntu 가상 머신을 새로 프로비저닝하고 소스를 클린 체크아웃해 공식 JDK 위에서 빌드하므로, 개인 환경 편향을 배제한 재현 가능성을 보장한다.

필수 상태 확인 부재

상태 확인을 강제하지 않으면 테스트가 깨진 PR도 실수나 임의 판단으로 병합될 수 있다. 결함 코드가 `main`에 흡수되면 이를 내려받은 팀원 전원의 로컬 빌드가 함께 깨지고, CD 파이프라인이 연결되어 있으면 결함 이미지가 운영 서버로 자동 배포되어 장애로 직결된다.

모범 답변

Mock과 Spy

Mock은 대상 클래스의 껍데기만 생성한 가짜 객체로, 스테빙하지 않은 메서드는 `null`, `0`, `false` 같은 기본값만 반환하고 실제 로직을 실행하지 않는다. Spy는 실제 객체를 감싼 래퍼여서 스테빙한 메서드만 가짜 응답을 반환하고 나머지는 실제 로직을 수행한다. 결제나 메일 발송처럼 실제 호출되면 안 되는 의존성은 Mock을, 레거시 클래스의 일부 동작만 대체하거나 호출을 감시할 때는 Spy를 쓴다.

커버리지의 한계

커버리지는 해당 라인이 실행되었는지만 측정하고 올바른 검증 로직으로 확인했는지는 측정하지 않는다. 검증문(Assertion) 없이 메서드를 호출만 해도 커버리지는 100%로 집계된다. 또한 경합 상태, 네트워크 타임아웃, 대규모 부하, 잘못된 요구사항 자체는 수치와 무관하므로 커버리지는 최소 안전선일 뿐 검증의 질과 경계값 검증이 더 중요하다.

모범 답변

Jenkins를 고수하는 이유

가장 결정적인 요인은 데이터 주권과 망분리 규제다. 금융, 의료, 방산 등에서는 소스코드와 빌드 산출물, 사내 자격 증명의 외부 SaaS 반출이 금지되는 경우가 많다. Jenkins는 인터넷이 차단된 사내 폐쇄망이나 에어갭 환경에서도 단독 구동하며, 사내 배포 시스템 연동과 전용 하드웨어 제어, 레거시 인증 연동을 플러그인과 스크립트로 자유롭게 확장할 수 있다.

CI Green이후의 장애

단위 및 슬라이스 테스트는 단일 스레드와 모의 환경에서 기능적 정확성만 검증하므로 커넥션 풀 고갈, 스레드 경합, 락 충돌, GC 병목, I/O 포화 같은 비기능 결함을 잡지 못한다. 배포 전 운영과 유사한 환경에서 Master-Worker 분산 부하 아키텍처로 검증하되, CI 연계와 메트릭 상관 분석에는 Grafana k6, 복잡한 사용자 행동 모델링에는 Locust, 레거시 프로토콜과 GUI 설계에는 Apache JMeter를 선택한다.

깊이 알아보기

GitHub Actions 워크플로 문법

<https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions>

Job 정의와 실행 의존성

<https://docs.github.com/en/actions/using-jobs/using-jobs-in-a-workflow>

보호 브랜치와 필수 상태 확인

<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>

Grafana k6 부하 테스트

<https://grafana.com/docs/k6/latest/>

깊이 알아보기

Spring Boot 테스트 레퍼런스

<https://docs.spring.io/spring-boot/reference/testing/index.html>

JUnit 5 사용자 가이드

<https://docs.junit.org/current/user-guide/>

AssertJ 문서

<https://assertj.github.io/doc/>

Gradle JaCoCo 플러그인

https://docs.gradle.org/current/userguide/jacoco_plugin.html



- EOF -